

Концепция

0. Core Concept

Activators — активаторы личности

Определение

Activators — это люди и практики, которые запускают процессы внутреннего движения, роста и трансформации личности.

Это могут быть:

- организаторы мероприятий,
- ведущие групповых практик,
- преподаватели и наставники,
- терапевты, коучи, фасилитаторы,
- проводники индивидуальных сессий,
- создатели пространств для телесной, эмоциональной, когнитивной и социальной активации.

Объединяет их не формат и не профессия, а **функция**:

они активируют состояние, внимание, навык, осознанность, потенциал.

Почему именно *Activators*

Традиционные термины (“организатор”, “провайдер услуг”, “ведущий”) описывают **роль**, но не суть.

Activator описывает:

- воздействие, а не иерархию;
- процесс, а не услугу;

- соучастие, а не обслуживание.

Activator не “продаёт событие” —
он **создаёт условия**, в которых человек может:

- войти в новое состояние,
- научиться,
- прожить опыт,
- восстановиться,
- раскрыться,
- синхронизироваться с другими.

Activator как субъект платформы

В архитектуре платформы **Activator** — это:

- первичный субъект предложения ценности;
- носитель практики, метода или пространства;
- источник Activities (активностей).

Activator может действовать:

- индивидуально (1:1 сессии, коучинг, терапия),
- в малых группах,
- в больших форматах (мероприятия, фестивали, церемонии),
- регулярно или разово,
- офлайн, онлайн или гибридно.

Activity как проявление Activator

Activity — это конкретное проявление Activator'а во времени и пространстве.

- Activator = *кто активирует*
- Activity = *как и когда происходит активация*

Один Activator может создавать:

- множество Activities,
- разных форматов,
- для разных возрастов, языков и контекстов.

Таким образом:

- платформа **не каталог событий**,
- и не маркетплейс услуг,
- а **карта активирующих практик и людей**, доступных сообществу.

Пользователь в этой модели

Пользователь платформы — не “клиент”, а:

- участник,
- исследователь,
- со-настройщик своего опыта.

Поиск на платформе — это не “что купить”, а:

какая активность и какой Activator резонируют с моим текущим состоянием и задачей.

Архитектурное следствие (важно)

Эта концепция напрямую влияет на архитектуру:

- **Identity важнее бренда**
(Activator как профиль, не как рекламная страница)
 - **Reputation и trust важнее продвижения**
(позже: социальные сигналы, рекомендации, on-chain репутация)
 - **Activities — это не товары,**
а точки входа в опыт (что оправдывает сложную модель расписаний, языков, возрастов, сопровождения и т.д.)
 - **Search — это поиск резонанса,**
а не фильтрация прайс-листа
(что оправдывает RAG, векторы и объяснимость)
-

Формулировка для начала документации (готовая)

Activators — это активаторы личности: люди и практики, которые создают условия для внутреннего движения, обучения, восстановления и трансформации.

Платформа объединяет Activators и их Activities, позволяя участникам находить резонансные форматы, состояния и сообщества — через смысловой поиск, а не через маркетинг.

Техническое задание (Product Requirements Document)

Роль: Chief Product Officer

Продукт: GPT-интерфейс для сбора, нормализации и поиска мероприятий/занятий через RAG

1. Цель продукта

Создать систему, в которой:

- пользователи **легко добавляют информацию о мероприятиях и занятиях** в любом удобном формате;

- информация **автоматически приводится к единому семантическому стандарту**;
- пользователи **ищут и получают релевантные результаты** через диалог с Custom GPT;
- все ответы GPT **строго основаны на данных, хранящихся во внешнем RAG-хранилище**.

GPT используется **исключительно как интерфейс взаимодействия**, а не как источник истины.

2. Целевая аудитория

Основные пользователи

- организаторы мероприятий (кружки, мастер-классы, воркшопы, студии)
- преподаватели и наставники
- родители / участники, которые рекомендуют занятия
- модераторы / кураторы экосистемы (опционально)

Ключевые ограничения аудитории

- низкая мотивация к заполнению форм
 - низкая терпимость к «сложным интерфейсам»
 - привычка публиковать информацию сначала в соцсетях (Facebook приоритет)
-

3. Основные сущности продукта (на уровне смысла)

3.1. Activity (Мероприятие / Занятие)

Обобщающая сущность, включающая:

- разовые события

- регулярные занятия
- образовательные и развивающие активности

3.2. Source (Источник)

Любая форма первичного ввода:

- текст
- ссылка
- изображение (скриншот)
- комбинация вышеуказанного

3.3. Normalized Representation

Внутреннее семантическое представление Activity, пригодное для:

- векторизации
- поиска
- сравнения
- агрегации

4. Функциональные требования

4.1. GPT как пользовательский интерфейс

4.1.1. Общие требования

- Custom GPT является **единственной точкой взаимодействия** для пользователя
- пользователь **не взаимодействует напрямую** с базой данных или веб-приложением

- GPT:
 - принимает ввод
 - интерпретирует намерение
 - инициирует соответствующие действия во внешней системе

4.1.2. Ограничения GPT

- GPT не хранит данные
 - GPT не является источником истины
 - GPT не отвечает без обращения к RAG, если вопрос предполагает фактический ответ
-

4.2. Добавление данных (Ingestion Flow)

4.2.1. Форматы ввода

Система обязана принимать:

- свободный текст без структуры
- одну или несколько ссылок
- изображения (скриншоты из соцсетей)
- комбинации форматов в одном сообщении

4.2.2. Поведение системы при добавлении

При любом вводе:

1. GPT определяет, что пользователь **передаёт информацию о мероприятии**
2. GPT извлекает:
 - описание
 - предполагаемый тип активности

- ключевые параметры (если присутствуют)
- 3. Данные передаются во внешнее веб-приложение для:
 - обработки
 - нормализации
 - сохранения
 - векторизации

4.2.3. Требования к пользовательскому опыту

- пользователь **не обязан**:
 - знать структуру
 - подтверждать поля
 - заполнять формы
- допустим сценарий:

«Вот ссылка, добавь»

«Скрин с ФБ, разберись»

4.3. Нормализация и приведение к стандарту

4.3.1. Цель нормализации

Привести разрозненные данные к:

- единому семантическому представлению
- пригодности для RAG
- сопоставимости между разными источниками

4.3.2. Требования

- нормализация выполняется **вне GPT**
- результат:
 - текстовое описание
 - структурированные смысловые блоки
 - embedding для векторного поиска

4.3.3. Обработка неполных данных

- система **должна принимать неполноту**
 - отсутствие части данных **не блокирует сохранение**
 - допускается постепенное обогащение сущности со временем
-

4.4. Поиск и Retrieval через GPT

4.4.1. Пользовательский поиск

Пользователь может:

- задавать вопросы в свободной форме
- описывать потребность, а не фильтры
- использовать субъективные формулировки («что-то спокойное», «для ребёнка 7 лет» и т.п.)

4.4.2. Поведение GPT при поиске

1. GPT интерпретирует запрос
2. GPT формирует retrieval-запрос к векторной базе
3. Получает релевантные Activity
4. Генерирует ответ **исключительно на основе полученных данных**

4.4.3. Ограничения

- **GPT не должен добавлять:**
 - несуществующие мероприятия
 - неподтверждённые факты
 - если данных нет — GPT сообщает об отсутствии релевантных результатов
-

5. Нефункциональные требования

5.1. Надёжность

- данные не теряются при ошибках GPT
- повторные попытки допустимы

5.2. Масштабируемость

- система рассчитана на рост количества:
 - пользователей
 - источников
 - мероприятий

5.3. Обновляемость

- возможна повторная обработка источников
 - возможна доиндексация при изменении стандартов
-

6. Критерии успешности (Product Success Criteria)

6.1. Для добавления данных

- пользователь может добавить мероприятие за ≤ 1 сообщение

- $\geq 80\%$ добавлений не требуют уточняющих вопросов

6.2. Для поиска

- пользователь получает релевантный ответ без знания структуры
- ответы воспроизводимы (одинаковый запрос → сопоставимые результаты)

6.3. Для продукта в целом

- GPT воспринимается пользователем как:
 - «умный приёмщик»
 - «понятный навигатор»
 - система не ощущается как форма или каталог
-

7. Явно не входит в текущий score

- ручная модерация пользователем
 - сложные фильтры и UI-панели
 - социальные функции
 - рейтинги и отзывы
 - платежи и бронирования
-

8. Принципиальная продуктовая позиция (как CPO)

Этот продукт:

- снимает когнитивную нагрузку с пользователя
- переводит хаос реального мира в машиночитаемую форму
- делает GPT не «мозгом», а интерфейсом к знанию

Архитектура решения

Activity — Unified Structure (event | service)

Activity Types: **event** and **service**

The Amanita platform models all offerings as **Activities**.

An Activity represents a structured public offering that can be discovered, reviewed, and published within the ecosystem.

While all Activities share a common lifecycle, policy gate, and discovery surface, **they differ fundamentally in how time, participation, and engagement are defined.**

To reflect this, the model introduces a single, explicit discriminator:

activity_type = event | service

This field is the **primary source of behavioral and structural differentiation** across the system.

Event Activities

An **event** represents a time-bound or schedule-defined collective experience.

Typical examples include:

- workshops and masterclasses,
- recurring classes or clubs,
- performances, ceremonies, and gatherings,
- one-time or repeating group events.

The defining characteristics of an **event** are:

- **explicit time structure** (single occurrence or recurrence),
- **shared participation** (one occurrence, many participants),
- **schedule-driven discovery** (“what happens this week / next month”).

In the model, events are characterized by:

- a formal schedule (dates, times, recurrence rules, exceptions),
- event-level capacity semantics,
- duration derived from occurrences,
- optional ticketing or event-page links.

Time is the organizing axis of an event.

Service Activities

A **service** represents an individual or on-demand offering that is not defined by a fixed event schedule.

Typical examples include:

- individual therapy or coaching sessions,
- bodywork or healing practices,
- tutoring, mentoring, or private instruction,
- consultations or personal sessions of various kinds.

The defining characteristics of a **service** are:

- **absence of event-style scheduling,**
- **one-to-one or small-unit participation,**
- **availability-driven engagement** (“by appointment”, “on request”).

In the model, services are characterized by:

- availability and booking policies instead of schedules,
- session-based participation semantics,
- duration options rather than fixed event length,

- explicit booking or contact methods.

Time exists, but only as **availability**, not as published occurrences.

Why This Is a Model-Level Differentiation

The distinction between **event** and **service** is **not a UI concern and not a minor field variation**.

It affects:

- how time is represented,
- how capacity is interpreted,
- how pricing is structured,
- how search and filtering work,
- which fields are required at review and publication stages.

Attempting to represent services as “events without dates” or events as “services with slots” leads to ambiguity, validation complexity, and broken discovery logic.

By introducing **activity_type** as a first-class discriminator:

- the model remains explicit and predictable,
 - existing event logic remains intact and unchanged,
 - service-specific logic can evolve independently,
 - future extensions (booking, tokens, NFTs) can attach cleanly.
-

Unified Lifecycle, Divergent Semantics

Both activity types:

- share the same lifecycle (**Draft** → **Review** → **Approved** → **Published**),

- pass through the same policy gate (КоныРода),
- appear in the same discovery surface,
- are governed by the same privacy and compliance rules.

They differ **only where semantics truly diverge**:

- timing,
- participation,
- booking and engagement.

This approach preserves architectural clarity while allowing the platform to represent a wide range of human activities without forcing them into an artificial event-only model.

0) Identity & Lifecycle (общие для всех)

- **activity_id** (уникальный идентификатор)
 - **activity_type**: **event** | **service**  *ключ дифференциации*
 - **status**: **Draft** | **SentToReview** | **Approved** | **Published**
 - **versioning** (черновики/версии, audit trail)
 - **creator/owner reference** (псевдонимный идентификатор Activator; без PII)
 - **timestamps** (created/updated/published)
-

1) Core Description (общие для всех)

- **title**
- **short_summary**
- **full_description**
- **format** (контролируемый список + “other”)

- **categories / taxonomy** (2 уровня + fallback user suggestion)
 - **age_groups** (babies...seniors)
 - **parental_accompaniment** (allowed|required|optional — для детских)
 - **language_requirements**
 - mode: irrelevant | understand_only | speak_and_understand | mixed
 - languages_to_understand[]
 - languages_to_speak[]
 - **media & external links**
 - official site
 - social links (enum)
 - (event/service-specific links — см. ниже)
 - **policy notes** (опционально: дисклеймеры/ограничения, без клиники)
-

2) Delivery & Location (общие, но с нюансом)

- **delivery_mode**: in_person | online | hybrid
 - **location_info**
 - city / area
 - venue (если применимо)
 - online platform / link (если применимо)
 - **service_area** (⚠ чаще нужно для service)
 - radius / districts / travel notes (*может быть общим полем, но заполняется обычно в service*)
-

3) Timing (главная точка дифференциации)

Если **activity_type = event**

- **event_timing**
 - schedule model (fixed dates / recurring)
 - exceptions / overrides
 - ability to compute next occurrence

Если **activity_type = service**

- **service_timing**
 - availability_type: by_request | fixed_windows | (future) bookable_slots
 - optional availability windows (если фиксированные)
 - booking policy (как записаться)

✓ Правило: *event_timing* и *service_timing* взаимоисключающие.

4) Participation & Capacity (дифференциация смысла)

Event

- **event_capacity**
 - group capacity / seats
 - (optional) min/max participants

Service

- **service_participation**
 - session_mode: one_to_one | family | small_group

- concurrent_clients (обычно 1)
 - practitioner-to-client model (без персонализации)
-

5) Duration & Pricing (частично общие, но структура разная)

Event

- **event_duration**
 - derived per occurrence OR fixed duration
- **event_pricing**
 - ticket price / donation / free
 - price range

Service

- **service_duration_options**
 - 30/60/90/custom etc.
 - **service_pricing_model**
 - per_session | per_hour | per_package | donation | free
 - optional package definition
-

6) Booking / CTA (call-to-action) (дифференциация)

Event

- **event_cta**
 - event page link

- tickets link (optional)

Service

- **service_cta**
 - booking_url (Calendly/сайт/форма)
 - or contact via public channel (social link)
 - or “Amanita booking (future)”
-

7) Source & Provenance (общие для всех)

- **sources**
 - canonical_url (если есть)
 - source_type (manual/link/screenshot/pdf)
 - (optional) raw_asset_ref (TTL)
 - **dedup hints** (possible duplicates, update vs new)
-

8) Review Metadata (общие для всех)

- **review_submission**
 - submitted_at
 - notes (опционально)
- **policy_gate_result**
 - approved/rejected/needs_clarification
 - reasons (структурировано)
 - policy_ref (КонныРода version)

